

Computer Vision for Game Piece Detection

Proposal by Kyla Kaplan (kylabkaplan@gmail.com)

Mentors: Oleksandr Zaitsev (oleksandr.zaitsev@cirad.fr),

Etienne Delay (etienne.delay@cirad.fr), Christophe Le Page (christophe.le_page@cirad.fr),

Pierre Bommel (bommel@cirad.fr)

Introduction/Synopsis

This project aims to integrate a real-time computer vision system with Cormas, the agent-based modeling platform developed in Pharo, to detect and track physical board game pieces. Using a webcam and a Python-based vision system, piece positions will be mapped to the Cormas grid in real-time. This will allow users to manipulate a tangible board and see their changes reflected directly in a simulation—ideal for educational settings and interactive modeling. The work involves first designing a calibration system, then implementing a reliable object detection pipeline, and finally establishing communication with Cormas via JSON file-passing.

In the context of Google Summer of Code, this project is a direct contribution to the open-source Cormas platform. All work will be committed to public repositories, extending Cormas with a new computer vision plugin. The implementation will serve as a foundation for a more generic “gaming interface” within Cormas — enabling users to interact with simulations through real-world physical components. This not only opens up new educational and interactive uses of Cormas but contributes code and documentation back to the Pharo and Cormas communities.



Benefits to the Community

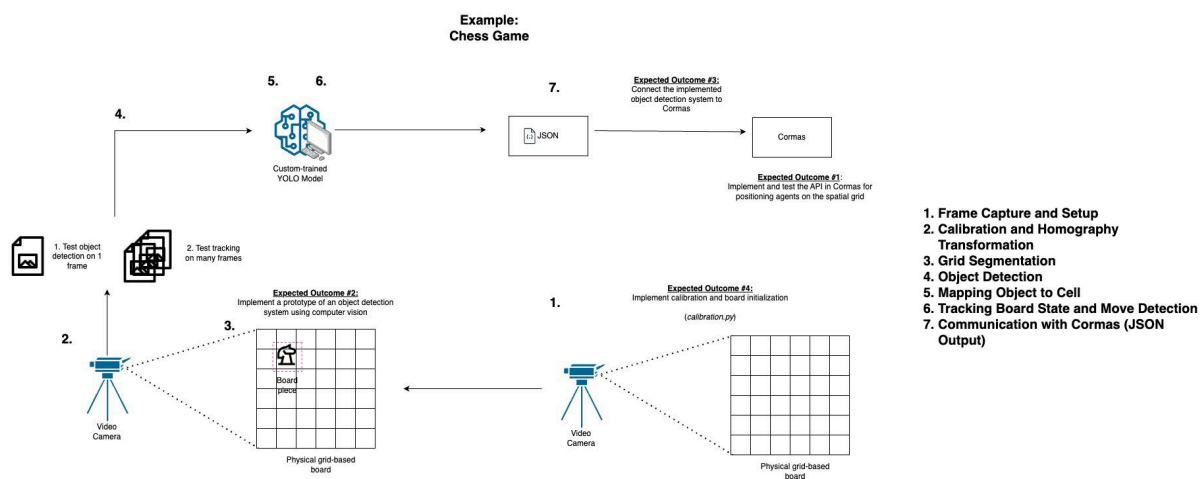
This project brings physical interaction into the realm of virtual simulation, extending the applicability of Cormas in educational, research, and workshop settings. By enabling tangible control over agent-based models, it lowers the barrier to entry for non-programmers, students, and facilitators. The integration of a practical and reusable computer vision pipeline also lays the groundwork for future smart board experiences within the Pharo ecosystem. The system is specifically well-suited for grid-based games and simulations where spatial position and piece identity are essential — such as Chess, Checkers, Go, Connect Four, or agent-based environments like cellular automata and territorial dynamics. Importantly, this project contributes a reusable vision module to the Cormas codebase, strengthening Cormas as an open-source framework for simulation and human interaction.

Deliverables

- Grid calibration tool
 - Board initialization and real-time updates
 - Object detection prototype with OpenCV
 - JSON communication layer
 - Cormas API for positioning objects/agents
 - Full demo with docs and tutorial
-

Proposed Technical Implementation

I will explain my proposed technical implementation step-by-step starting from the camera input to Cormas.



This project will start with developing a Python-based computer vision pipeline to detect and track physical board game pieces (using specifically chess as a prototype) and export their positions for the Cormas simulation. The system uses a fixed camera feed of the board and processes it in real time to identify which pieces occupy which board squares. The implementation emphasizes modularity so that while an 8x8 chessboard is the initial case, the solution can be generalized to other board sizes, piece types, and layouts. The pipeline consists of the following stages: frame capture, board calibration and homography, grid segmentation, piece detection with YOLO, mapping detections to grid cells, state change tracking, and JSON output for integration with Cormas.

All implementations—ranging from the Python-based vision system to the Pharo-side communication layer—will be designed as open-source modules. In particular, the JSON-based interface and the Cormas API extension will be developed with reusability in mind, forming the base of a generic gaming interface for Cormas models. The long-term goal is for any Cormas model to be easily extended into a real-world, physically interactive simulation using this work.

A few anticipated limitations and future considerations that could possibly be implemented within the scope of this project are listed at the end of this section.

1. Frame Capture and Setup

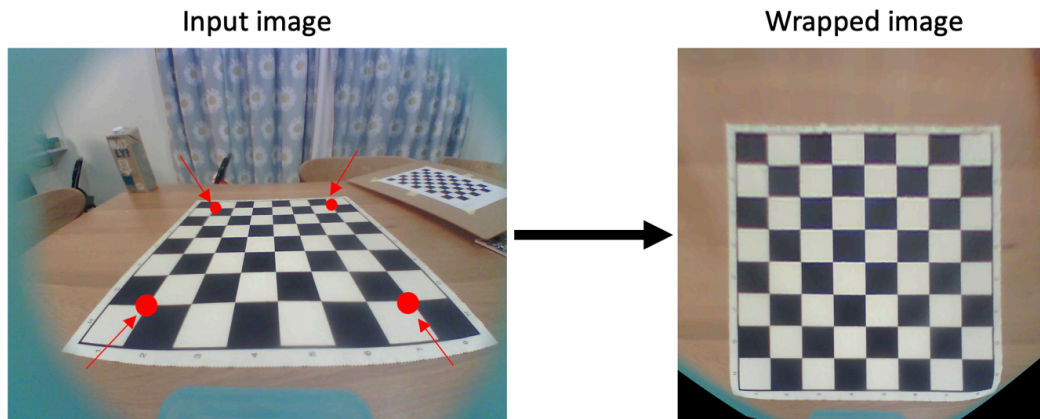
The pipeline begins by capturing video frames from a fixed camera using OpenCV. We assume a stable camera mounted such that the entire board is visible in the frame. A stable viewpoint is important because calibration (done once at the start) will be invalid if the camera moves. Using `cv2.VideoCapture`, the system will grab frames at a 1 frame-per-second (1 Hz) rate (chess moves occur relatively slowly, so 1 fps is sufficient to catch each move). Each frame is read as a BGR image (at a resolution that clearly shows the pieces, e.g. 720p+). Basic preprocessing will be applied here, such as resizing the frame or correcting color/brightness, but the main preparation is done in the calibration stage.

At startup, the system will prompt the user for a one-time manual calibration. The first captured frame (showing the empty or initial board) is displayed in an OpenCV window, and the user is asked to click the four corners of the board (while it's empty). These four 2D points in the image represent the board's outer corners in the camera view. This manual calibration step establishes the mapping between the camera's perspective and the logical board coordinates. Once the corners are obtained, the calibration interface can be closed and the points proceed to the homography computation.

2. Calibration and Homography Transformation

Using the four corner points from the user, we compute a homography matrix **once** (a perspective transform) that will map the camera's view of the board to a top-down, rectified view. In practice, we define the destination coordinates for these four corners as an 8×8 grid's corners. For example, if we choose a normalized 800×800 pixel output for the board, the four clicked corners are mapped to (0,0), (800,0), (800,800), and (0,800) in the destination image (corresponding to one corner of each of the 4 outer squares of the chessboard).

We then call OpenCV's `cv2.getPerspectiveTransform` or `cv2.findHomography` to compute the 3×3 transformation matrix H using those source and destination points. This matrix H encapsulates the perspective warp needed to flatten the board image.



With the homography matrix computed, the system applies it to each incoming frame to generate a top-down view of the board (see image example above)¹. We use `cv2.warpPerspective` to transform the original camera frame by H , producing a new image where the chessboard is directly facing the camera (correcting for tilt or perspective). The output of `warpPerspective` is set to a fixed size (for instance, 800×800 pixels) so that each square on the board occupies a consistent pixel region in this image. This warping essentially normalizes the coordinate space: an 8×8 grid in the warped image corresponds exactly to the 8×8 chessboard. It also crops out the rest of the scene (only the board area is seen in the warped image), which reduces false detections outside the board. The homography need only be computed once per session (unless the camera or board moves); it will be reused for all frames, ensuring consistency in mapping coordinates. We will store this matrix and the defined output size for use in subsequent steps. By calibrating

once and warping each frame, we greatly simplify the problem of locating pieces – the board is now aligned with a fixed grid, making it much easier to determine which square a piece is on.

3. Grid segmentation

After obtaining the perspective-corrected top-down image of the board, the next step is to divide this image into the 64 individual squares of the chessboard grid. Because the warped board image has a known resolution (e.g. 800×800) and represents an 8×8 grid, each square occupies an equal sub-region (in this example, 100×100 pixels per cell). We segment the board by calculating the boundaries of each cell: essentially, we draw the grid lines at every 1/8th of the width and height.

We will maintain a mapping of these cell regions to their logical names. For example, we know the array indices [0][0] in our 8x8 grid correspond to square A1, [1][0] to B1, ..., [7][7] to H8 (assuming one axis for columns A–H and one for rows 1–8). The exact indexing depends on the orientation decided during calibration (if the user clicked the corners in a known order, we can assign which corner is A1 vs H8, etc.). In code, we can store a list of cell corner coordinates or simply infer cell index by dividing coordinates by the cell width/height. This mapping function will take any pixel coordinate (in the warped image space) and return the board cell (for instance, (x=250, y=450) → E5, depending on calibration).

4. Object detection

Considering chess as the prototype example: there are 12 classes in total for standard chess (6 piece types × 2 colors). For detecting the game pieces on the board, we will leverage a YOLO object detection model specialized for chess pieces.

YOLO processes the image in a single forward pass of the neural network and returns a set of detected bounding boxes, each with a confidence score and a class label (e.g. “black_knight” with 95% confidence at a certain pixel location). Non-maximum suppression is automatically applied by the model to filter out duplicate detections of the same piece. We will set an appropriate confidence threshold (for instance, 0.5) to ignore very low-confidence detections, reducing false positives.

The output of the detection step for each frame will be a list of pieces detected, along with the coordinates of their bounding boxes. For example, YOLO might return something like: White_King at (x1, y1, x2, y2), Black_Pawn at (x1, y1, x2, y2), ... for all pieces it sees. Because we are analyzing a chessboard where each square has at most one piece, we expect at most 32 pieces in the list (the maximum number of pieces on board at game start). The YOLO model’s speed is sufficient that even on modest hardware it can process ~5–10

frames per second; running it at 1 fps ensures plenty of processing headroom, and it could potentially be increased if needed.

5. Mapping Object to Cell

The next task is to map each detected piece to a specific board cell using the earlier calibration information. The simplest method (given that we warped the image to a fixed grid) is to use the center point of each bounding box in the warped image and determine which cell coordinates it falls into.

For instance one approach could be the following: for each detected piece's bounding box, we compute its center (cx , cy) in pixel coordinates relative to the top-down board image. Then, using the grid cell size (e.g. 100 pixels if $800/8$), we determine the column index as $col = \text{floor}(cx / \text{cell_width})$ and the row index as $row = \text{floor}(cy / \text{cell_height})$. For example, if $cx=250$, $cy=750$ in an 800×800 image, that yields $col=\text{floor}(250/100)=2$ and $row=\text{floor}(750/100)=7$. If we mapped col 2 $\rightarrow$ "C" and row 7 $\rightarrow$ "8", that coordinate corresponds to C8 on the chessboard. We then know the detected piece (say "black bishop") is at cell C8. This calculation is done for every detection. Importantly noting however, that this methodology could most likely change during implementation.

Either way, because the homography aligned the board, this mapping is straightforward – even if the original camera view was angled, the warp ensures that a piece's center lands more or less in the correct cell area. If previous attempts of simplistic mapping have failed, we might have to include an extra calibration step at the beginning.

6. Tracking Board State and Move Detection

With each new frame's detections converted into board positions, the system can maintain a record of the board state over time and thereby detect moves. We will represent the current state as a data structure (for example, an 8×8 array or a dictionary keyed by cell) that holds which piece (if any) occupies each square. After processing a frame and mapping all detected pieces to cells, we obtain the current state. We then compare this against the previous state (from the last processed frame, or an initial state if it's the first move).

I propose to use the following tracking algorithms BoTSort and ByteTrack for multi-object tracking (MOT), in companion with YOLOv8, as these typically go hand-in-hand. Typically these algorithms handle occlusions well, which would be one of the largest challenges in a board game-tracker (e.g. hand covering board during piece movement). We will make use of the tracking algorithm's outcome to adjust our board game state and more importantly

accommodate for pieces removed permanently from the board (i.e., defeated during the game). Here, thresholds within the tracking algorithms can be applied to determine for how many frames we keep object IDs in memory. Evidently, we need to test and find the optimal threshold during a live game implementation (see last part).



7. Connect to Cormas

After processing each frame and updating the board state, the pipeline will output the results in a structured JSON format. The JSON will contain all currently detected pieces and their logical positions (e.g., "white_knight": "F3" or a list of piece objects with properties). This output is generated once per frame (once per second) so that external systems can ingest near-real-time updates of the board.

This would represent the full board state at that time. Alternatively, we could structure it as a dictionary of cells to piece, but a list of pieces is straightforward for the simulation to parse. Each piece entry includes its type (and color) and the cell (in chess notation) it occupies. If a cell is empty, it simply won't appear in the list.

For integration with Cormas, the idea is that the Cormas simulation platform can consume these JSON outputs to update the state of the simulation model. I will implement the JSON export by writing to a file (or a series of files) in a known directory.

Care will be taken to ensure the write happens after each frame's processing, and we might include a small delay or check to avoid writing too frequently if not needed. At 1 fps, this is

manageable. It will also handle cases where no piece is detected (e.g., if the board is momentarily empty or occluded) by perhaps skipping output or indicating an empty state, though in normal operation the board will have pieces.

Ambitiously, the Cormas library can have presets on calibrations, but as a first working prototype, it should visualize in its spatial grid the real-time updates.

8. Test Iterations

To ensure reliability and robustness of the system, especially regarding practical usability and edge cases, I will conduct multiple test iterations using a human testing group. Specifically, participants (selected peers and mentors) will perform at least three full chess rounds, interacting with the physical board under realistic conditions. During these tests, the system's accuracy in detecting pieces, recognizing moves, maintaining consistent calibration, and reliably communicating with the Cormas simulation will be evaluated. The testers will also intentionally attempt scenarios likely to reveal issues, such as partially obscuring the board, making rapid or subtle piece movements, or introducing non-standard piece positions. Their feedback, observations, and recorded gameplay sessions will be collected and analyzed to identify and troubleshoot real-world problems, leading to targeted refinements of the detection pipeline and calibration processes. This iterative, human-centric testing approach will significantly enhance the practicality and dependability of the final implementation.

Similarly, the API communication with Cormas will also be extensively tested according to Test-Driven Development (TDD) practices.

There are a few important consequences and considerations keeping in mind the specifics of this implementation:

- **Board Size and Layout:** The calibration step can accommodate any rectangular board – the user clicks the four corners of the playable area, which could just as easily be a 10×10 board or a 19×19 Go board. The homography and grid segmentation will be adapted by parameterizing the number of rows and columns.
- **Arbitrary Piece Types:** The object detection component (YOLO) can be retrained or swapped out to detect different pieces. The pipeline is modular, so the detection step would load a model and class labels appropriate to the game in question.
- **Performance Considerations:** We could also skip warping every frame by mapping detections via the homography matrix mathematically (to save the cost of `warpPerspective` each time), but since 1 FPS is low, this is not a bottleneck. Still, the option exists to optimize if needed.

- **User Interface and Experience:** For general use, we might extend the calibration UI to allow the user to input the grid size (rows/columns) and perhaps even save calibration profiles. For example, a user could calibrate once for their chessboard and save the homography matrix to reuse in future sessions without re-calibrating, as long as the camera and board setup remain the same. Generalizing this, profiles for different games can be stored (with the board dimensions and a corresponding YOLO model file for that game).
- **Camera position:** We will begin with a manual calibration mode, where the user clicks the four corners of the board to establish accurate mapping to the logical grid. This approach ensures robust calibration even for non-standard or colored boards. If feasible, we will explore automating the calibration process using image processing techniques, which would reduce user intervention and further streamline the setup.
- **Time-of-Flight (TOF) Sensor:** Incorporating a TOF sensor, such as those found in the Azure Kinect, can significantly enhance detection accuracy by providing precise depth information. This depth data can improve the calibration process by offering an additional measure of the board's orientation and distance from the camera, especially in challenging lighting conditions or with reflective/non-uniform surfaces. Moreover, TOF data can aid in distinguishing between the board and the background, helping to filter out noise and improve the piece detection pipeline. Integrating TOF capabilities represents a valuable optimization that, if feasible, could further streamline the setup and ensure more reliable mapping and object detection.

Proposed Timeline

I plan to implement this project over an extended 22-week timeline, in line with GSoC guidelines. This schedule will provide sufficient time to develop, test, and refine each component—from initial setup, manual calibration, object detection, and tracking to full integration with Cormas via JSON communication. The extended timeline also allows for iterative improvements, exploration of advanced features such as automated calibration and TOF sensor integration, and comprehensive testing in real-world scenarios, ensuring the final system is robust, accurate, and user-friendly.

Period	Tasks & Milestones
Community Bonding	- Connect with mentors - Complete Pharo MOOC and Advanced Pharo MOOC - Review Cormas and Pharo internals - Set up development environment

Week 1–2	- Set up the Python environment and OpenCV framework - Implement frame capture (1 fps) and basic preprocessing - Develop a manual calibration interface (user clicks four board corners)
Week 3–4	- Compute the homography matrix from user-selected corners - Apply <code>cv2.warpPerspective</code> to obtain a top-down view of the board - Segment the warped image into grid cells and validate the mapping
Week 5–6	- Integrate the object detection pipeline by loading a pre-trained YOLO model specialized for chess pieces - Run YOLO on warped frames to generate bounding boxes and piece labels - Tune detection confidence thresholds
Week 7–8	- Map detected bounding box centers to corresponding grid cells using the known cell dimensions - Develop a data structure to represent board state (e.g., an 8×8 array or dictionary keyed by cell)
Week 9–10	- Implement multi-object tracking (e.g., BoTSort and ByteTrack) to handle occlusions and overlapping detections - Refine tracking algorithms to address transient overlaps and ensure robust move detection
Week 11–12	- Develop a JSON export pipeline that outputs the full board state (piece type, color, grid cell) once per frame - Begin prototyping integration with a Cormas API via JSON file-passing
Weeks 13–14	- Conduct initial integration testing between the Python pipeline and Cormas - Run test sessions with a human testing group to gather feedback on calibration, detection, and tracking performance
Weeks 15–16	- Explore advanced features: automate calibration using image processing techniques and evaluate TOF sensor integration (e.g., with Azure Kinect) for improved depth accuracy
Weeks 17–18	- Optimize performance by experimenting with alternative mapping (e.g., applying the homography mathematically instead of warping each frame) - Further refine tracking algorithms based on test results
Weeks 19–20	- Conduct extensive integration testing between the Python pipeline and the Cormas simulation - Iterate on UI refinements, calibration profile storage, and error handling based on user feedback
Weeks 21–22 (Final)	- Finalize comprehensive documentation, tutorials, and user guides - Prepare and deliver the final demo, final report, and blog post

Evaluation)	- Conduct a final review with mentors and incorporate any last-minute improvements
-------------	--

Expected Outcomes

Upon completion of this project:

- Users will be able to physically move game pieces on a tangible board and observe immediate, synchronized updates within the Cormas simulation.
- The system will be able to detect objects given the respective training model and initialize the board accordingly.
- Calibration tools will provide accurate and straightforward board setup for further use-cases.
- The developed system will offer a robust, extendable framework for further hybrid simulations.

About Me

I am a first-year Master's student in Financial Technology & Computing at Università della Svizzera italiana (USI), with a BSc degree in Informatics with a focused background in both programming and AI. During my bachelor's thesis, I collaborated with a company that works closely with the "Glamorous Toolkit" team, which deepened my appreciation for exploratory programming environments and the power of Smalltalk-based systems. I've worked with OpenCV and Python during my undergraduate studies and have been learning the Pharo environment through the publicly available MOOCs, and by working in the field.

I enjoy building tools that bridge the digital and physical worlds, and this project aligns with my interests in user interaction, education, and intelligent systems. I'm committed to learning quickly, building responsibly, and contributing meaningfully to the Pharo open-source ecosystem in the long run. I've already begun learning how to contribute to the Pharo ecosystem through community events like isLoop 2025, where I worked alongside other developers on exploratory Pharo-based projects. Through this GSoC project, I aim to deepen that involvement by directly contributing code, tools, and documentation to Cormas, supporting its continued growth as an open-source platform.

Commitment

I will be largely available during the GSoC period and will dedicate 20–30 hours a week to this project. I will also be continuing working part-time (15 hours/week) during GSoC at a company developing in Smalltalk, which only keeps me more closely aligned with Pharo development practices. Over the extended timeline I can more than manage to complete all the expected outcomes.

Personal Information

Name: *Kyla Kaplan*

Email: *kylabkaplan@gmail.com*

GitHub: *github.com/kybeka*

Discord Username: *kyb3ka#5748*

LinkedIn: *https://www.linkedin.com/in/kyla-kaplan/*

University: *Università della Svizzera italiana (USI)*

Degree: *MSc of Informatics and Economics, Financial Technology & Computing*

Location & Time Zone: *Lugano, Switzerland – (GMT+2)*

A few sources used for research and inspiration

1. <https://blog.roboflow.com/automated-chess-game-recording-computer-vision/>
2. <https://opencv.org/>
3. <https://github.com/ultralytics/yolov5>
4. <https://developer.mamezou-tech.com/en/robotics/vision/calibration-pattern/>
5. <https://github.com/Rizo-R/chess-cv>
6. <https://www.raspberryturk.com/>
7. <https://devpost.com/software/chess-piece-detection#:~:text=Our%20project%20of%20cuses%20on%20implementing,with%20its%20corresponding%20class%20label>
8. <https://medium.com/tech-blogs-by-nest-digital/object-tracking-object-detection-tracking-using-bytetrack-0aaf924d292>
9. <https://docs.ultralytics.com/modes/track/#why-choose-ultralytics-yolo-for-object-tracking>
10. <https://muhammadrizwanmunawar.medium.com/ultralytics-yolov8-object-trackers-botsort-vs-bytetrack-comparison-d32d5c82ebf3>